INGENIERÍA Y CALIDAD DE SOFTWARE

Lean y Kanban

20 de octubre de 2026

1. Filosofía Lean y generación de valor

Lean es una filosofía de mejora de procesos orientada a generar valor para el cliente con el menor desperdicio posible. Surgió en la producción japonesa de la posguerra y luego se adaptó al desarrollo de software y a otros servicios basados en conocimiento. Su punto de partida es comprender el proceso real, identificar qué actividades aportan valor y revisar aquellas que consumen tiempo o recursos sin aportarlo. Una mejora local no sirve si perjudica al producto o al flujo completo.

Lean y el agilismo comparten la importancia de aprender a partir de la experiencia, entregar valor, colaborar y adaptar la forma de trabajo. Kanban y Scrum se distinguen principalmente por sus prácticas de gestión: uno organiza el trabajo como flujo continuo, mientras que el otro usa Sprints con una duración fija. Lean es una filosofía; Kanban es un método de gestión y mejora de servicios, y Scrum un marco de trabajo. Ninguno de ellos equivale a una secuencia universal de pasos para desarrollar software.

2. Los siete principios Lean

1. **Eliminar desperdicios.** Examinar las actividades del proceso para quitar las que no generan valor y reducir las que son necesarias pero consumen recursos. Conviene entender primero el trabajo y sus dependencias: eliminar un paso sin comprender su función puede empeorar el resultado.
2. **Amplificar el aprendizaje.** Hacer explícito el conocimiento que queda en las personas, compartirlo dentro del equipo y aprender de la retroalimentación. Evita que la organización dependa de una sola persona que sabe cómo funciona una parte del producto.
3. **Embeber la integridad conceptual.** Mantener coherencia y excelencia técnica en el producto. La calidad se construye durante el trabajo y no se negocia para aparentar que se entregó antes; prevenir defectos evita correcciones posteriores.
4. **Diferir compromisos hasta el último momento responsable.** Decidir cuando se cuenta con información suficiente y aún queda margen para actuar. No significa postergar indefinidamente: hay un punto a partir del cual no decidir también tiene consecuencias. En requisitos emergentes, especificar todos los detalles al inicio puede desperdiciar esfuerzo si cambian al recibir retroalimentación.
5. **Dar poder al equipo.** Permitir que quienes realizan el trabajo tomen decisiones sobre él. La autonomía requiere formación, motivación y responsabilidad por los compromisos asumidos; también se apoya en la colaboración y el conocimiento compartido.
6. **Ver el todo.** Comprender el producto, el problema del cliente y el proceso completo, en lugar de optimizar únicamente una tarea o una especialidad. Programar bien una parte aislada no alcanza si no se entiende qué necesidad de negocio resuelve.

7. **Entregar lo antes posible.** Completar y poner en manos del cliente trabajo valioso con la calidad requerida. La entrega temprana produce retroalimentación, reduce incertidumbre y permite corregir el rumbo con información real.

3. Desperdicios en el trabajo de software

Desperdicio es el esfuerzo que no genera valor. El **tipo 2 o desperdicio puro** puede evitarse o eliminarse; el **tipo 1 o desperdicio necesario** cumple una función indirecta que no se puede suprimir por completo y debe minimizarse. Por ejemplo, cierta coordinación o supervisión puede resultar útil para resolver obstáculos, aunque un exceso de niveles y aprobaciones alargue el trabajo.

Los desperdicios de la manufactura se reinterpretan en los servicios basados en conocimiento. La lista siguiente incluye los siete desperdicios clásicos y el talento no utilizado como agregado.

Desperdicio	Cómo aparece en software
Inventario	Trabajo parcialmente terminado: funcionalidades comenzadas que aún no cumplen la *Definition of Done* ni pueden entregarse. Ocupan capacidad, requieren retomar contexto y todavía no aportan valor al cliente.
Sobreproducción	Características extra o *gold plating*: implementar funcionalidades que no se pidieron o no se necesitan. *YAGNI* ayuda a evitar hacer algo solo porque quizá se use algún día. *KISS* apunta a resolver de manera simple; una característica extra puede ser simple y seguir siendo desperdicio.
Proceso extra	Pasos o controles sin utilidad en el contexto actual. Si se reemplaza una planilla por una aplicación, copiar todos los controles manuales de la planilla puede perpetuar trabajo que el propio software ya evita.
Transporte	Cambio de tareas y transferencia de conocimiento entre personas o áreas. Incorporar a alguien para reemplazar a una persona clave exige tiempo de capacitación y traspaso del contexto.
Esperas	Trabajo detenido por dependencias, autorizaciones, disponibilidad del cliente o revisiones de código pendientes. Una revisión entre pares puede ser valiosa, pero la demora en obtenerla sigue siendo tiempo de espera que conviene gestionar.
Movimiento	Cambio de contexto o tiempo de preparación al alternar proyectos, actividades, herramientas y espacios mentales. Es especialmente costoso cuando una persona divide su jornada entre desarrollo y soporte.
Defectos	Esfuerzo de corrección, repetición y atención de fallas que podrían haberse prevenido mediante mejores prácticas y calidad incorporada al proceso.
Talento no utilizado	Desaprovechar la experiencia, ideas o capacidad de decisión de las personas que hacen el trabajo.

El **foco** reduce el costo de alternar tareas: empezar una pieza pequeña, terminarla y luego tomar otra suele hacer avanzar más trabajo valioso que mantener muchas cosas abiertas. Para evaluar avance resulta más claro preguntar si una pieza cumple los criterios acordados de terminación que asignarle un porcentaje subjetivo de completitud. Lo iniciado y no terminado exige seguimiento, reconexión y transferencia sin constituir una entrega.

4. Kanban como método de gestión del flujo

Kanban, con mayúscula, designa un método para definir, gestionar y mejorar servicios, en especial trabajos basados en conocimiento. La palabra japonesa *kanban*, con minúscula, significa tarjeta o señal: identifica una pieza de trabajo o la necesidad de reponerla. En el tablero, una tarjeta permite saber qué hay que hacer y en qué estado se encuentra. En software pueden representarse historias de usuario, defectos, solicitudes de soporte, cambios y tareas de mantenimiento.

La idea central es que las piezas recorran un **flujo continuo** hasta su entrega, sin esperar a que termine un Sprint para reiniciar el tablero. Lo que finaliza es cada pieza de trabajo; el servicio sigue recibiendo y entregando solicitudes. El flujo debe ser ágil y predecible sin sacrificar calidad. Los valores asociados incluyen transparencia, equilibrio, colaboración, foco en el cliente, flujo, liderazgo, entendimiento, acuerdo y respeto.

Kanban parte del **proceso que efectivamente existe**: sus pasos, roles, responsabilidades y tipos de demanda. El tablero se configura de acuerdo con ese proceso; no hay un tablero idéntico para todos los equipos. Si dos tipos de trabajo requieren recorridos realmente incompatibles, pueden necesitar tableros diferentes. El método tampoco impone una tecnología de desarrollo ni prohíbe incorporar prácticas útiles de otros enfoques.

5. Principios de cambio y prácticas generales

Gestión del cambio y entrega de servicios

Los principios de **gestión del cambio** son: comenzar con lo que se hace ahora, entendiendo el proceso real y respetando roles y responsabilidades; acordar mejoras mediante cambios evolutivos; y fomentar actos de liderazgo en todos los niveles. Empezar desde la situación actual reduce la resistencia al cambio y permite comprobar con resultados si una modificación ayuda.

Los principios de **entrega de servicios** proponen comprender las necesidades y expectativas del cliente; gestionar el trabajo y permitir que las personas se organicen alrededor de él; y revisar periódicamente los servicios y sus políticas para mejorar los resultados. Estas decisiones se apoyan en transparencia, inspección y adaptación.

Las seis prácticas

1. **Visualizar el trabajo:** hacer visible el proceso, las tarjetas, sus estados, bloqueos y colas. Todos observan la misma información para coordinarse.
2. **Limitar el trabajo en progreso (WIP):** fijar cuántas piezas admite cada paso, según su capacidad, para evitar acumulación y saturación.

3. **Gestionar el flujo:** observar cómo avanzan las piezas, detectar cuellos de botella y favorecer la terminación antes de iniciar más trabajo.
4. **Hacer explícitas las políticas:** acordar criterios visibles para admitir, tomar, mover y terminar trabajo.
5. **Usar ciclos de retroalimentación:** establecer reuniones o cadencias adecuadas al servicio para revisar demanda, avance y problemas.
6. **Mejorar colaborativamente y evolucionar de forma experimental:** cambiar políticas o capacidad a partir de lo observado y comprobar el efecto de los cambios.

6. Tablero, colas, sistema Pull y límites de WIP

Cada columna del tablero representa un estado o paso relevante del proceso. Un flujo posible es **Pendiente → Análisis → Desarrollo → Pruebas → Entrega**. Algunos pasos separan **En progreso** de **Hecho/Listo**: esta segunda subcolumna funciona como cola para que el paso siguiente tome la tarjeta cuando tenga capacidad. La cola suele atenderse con un criterio FIFO (*first in, first out*), salvo que una política explícita determine prioridades o excepciones.

Pendiente → Análisis (en progreso / hecho) → Desarrollo (en progreso / hecho) → Pruebas (en progreso / hecho) → Entregado

En un sistema **Pull**, quien dispone de capacidad *toma* trabajo desde la cola anterior; la etapa previa no lo empuja directamente al estado "En progreso" de la siguiente. De ese modo la asignación ocurre cuando es posible trabajar en la tarjeta. El límite de **WIP (*Work in Progress*)** señala el máximo de piezas admitidas en un estado o paso. En un tablero que agrupa "En progreso" y "Hecho" bajo un mismo paso, el límite se cuenta sobre **ambas subcolumnas juntas**: no se gana capacidad ficticia por pasar tarjetas a "Hecho" si nadie de la etapa siguiente pudo tomarlas.

Los límites se establecen a partir de la capacidad real del paso y luego se ajustan de forma empírica. Un valor inicial puede orientarse por la cantidad de personas disponibles, pero no es una fórmula universal. Cuando una columna alcanza su límite, se deja de ingresar trabajo allí hasta liberar espacio. Mirar el tablero **de derecha a izquierda** ayuda a detectar qué entrega está trabada y qué etapa debe desocuparse primero. La consigna es **dejar de empezar y empezar a terminar**.

Un equipo saturado al 100 % pierde margen para absorber variaciones y ayudar a resolver bloqueos. Si una etapa está llena, una persona disponible puede sumarse a la tarea detenida o ayudar a la etapa siguiente (*swarming*), siempre que cuente con las habilidades necesarias. El objetivo es mejorar el flujo completo, no mantener a cada persona permanentemente ocupada.

El **límite de la cola de entrada** también merece una decisión explícita: admitir pedidos sin restricción crea expectativas y tiempos de espera potencialmente muy largos antes de empezar el trabajo. Limitar lo que se acepta o acordar claramente cuándo comienza el compromiso permite gestionar la demanda y las expectativas del cliente. Para que el flujo y sus métricas sean comparables, conviene dividir el trabajo en piezas relativamente pequeñas y de tamaños parecidos; con suficiente uniformidad, estimar cada tarjeta puede aportar poco valor.

7. Políticas explícitas y cadencias

Las **políticas** explicitan cómo funciona el servicio: condiciones para admitir una solicitud, cuándo puede pasar a la siguiente columna, criterios para tomarla, límites de capacidad y condiciones de terminación. Pueden incluir *Definition of Ready*, *Definition of Done*, revisiones de código requeridas antes de integrar cambios o la documentación mínima de una historia. Deben ser visibles, comprensibles, aplicables y revisables por quienes prestan el servicio.

Se pueden distinguir tipos de trabajo —funcionalidades nuevas, mantenimiento y correcciones de defectos— y observar qué parte de la demanda ocupa cada uno. Una distribución observada, por ejemplo 60 % de funcionalidad, 30 % de mantenimiento y 10 % de defectos, sirve para conversar sobre la capacidad necesaria; no constituye una proporción obligatoria para otros equipos. Las políticas también pueden fijar excepciones a FIFO o clases de servicio para solicitudes urgentes, de fecha fija, estándar o intangibles, cuidando que las excepciones no destruyan la previsibilidad del flujo.

Como el tablero **permanece activo**, hace falta decidir cuándo se reponen los pendientes: al quedar capacidad disponible, en una reunión periódica o mediante otra regla acordada. Puede haber una reunión breve de coordinación del equipo y una retrospectiva con la frecuencia que resulte útil; no están atadas al cierre de un Sprint. Estas cadencias crean oportunidades para revisar bloqueos, políticas y resultados.

8. Métricas de tiempo y eficiencia del proceso

Métrica	Intervalo observado	Qué permite ver
Lead Time	Desde que el cliente realiza la solicitud hasta que recibe lo solicitado.	Tiempo total vivido por el cliente, incluidas las esperas antes de comenzar y dentro del proceso.
Cycle Time	Desde que se empieza a trabajar la pieza hasta que queda terminada.	Tiempo de permanencia dentro del proceso una vez iniciado el trabajo; incluye pausas y colas internas.
Touch Time	Suma del tiempo en que efectivamente se trabaja sobre la pieza.	Tiempo activo, excluyendo períodos en que la pieza permanece esperando sin intervención.

Con esos límites temporales, **Touch Time ≤ Cycle Time ≤ Lead Time**. Si un pedido espera un año y se resuelve en una semana, el equipo puede registrar una semana de *Cycle Time*, mientras que el cliente experimenta un *Lead Time* de aproximadamente un año y una semana. Observar solo el tiempo interno escondería esa demora.

La **eficiencia del proceso** relaciona el tiempo de trabajo efectivo con el tiempo total transcurrido desde la solicitud hasta la entrega (*Elapsed Time*, tomado aquí como el *Lead Time*). Se deben usar las mismas unidades en numerador y denominador.

$$\textbf{Eficiencia del proceso} = \textbf{Touch Time} / \textbf{Lead Time} \times 100\ \%$$

Por ejemplo, si una pieza recibió 8 horas de trabajo efectivo y estuvo 80 horas desde la solicitud hasta la entrega, la eficiencia fue del 10 %. El valor ideal de 100 % implicaría ausencia total de esperas y normalmente no es alcanzable. La métrica ayuda a localizar demoras y desperdicios, sin convertir la ocupación permanente de las personas en un objetivo.